

TP 5 — Jointures, audit référentiel et entonnoir de conversion

Big Data Engineering — Master 1 — DMI / FST / UCAD

Séance 5 — Spark SQL avancé et analyse métier.

Consignes :

1. Complétez toutes les cellules marquées `# === À COMPLÉTER ===` (les `...` indiquent les trous) ;
2. Remplissez le **tableau de relevés** au fil des exercices ;
3. Rédigez les réponses aux questions de réflexion (cellules « *Votre réponse :* ») ;
4. Exécutez le notebook **de bout en bout** (« Restart & Run All ») avant de le pousser **avec ses sorties**.

Livrabale : ce notebook, dans `notebooks/` de votre dépôt GitHub, poussé **avant la séance 6**.

0. Vérification de l'environnement

Comme aux TP précédents : Python ≥ 3.9 , PySpark installé, données du fil rouge générées à l'**échelle 0.1** avec la **graine 42** (par défaut du script). Si le dossier `data/` est absent (ou après un reset Colab), dé-commentez la cellule de génération.

```
In [1]: import sys, platform
print("Python :", sys.version.split()[0], "-", platform.system())

import pyspark
print("PySpark :", pyspark.__version__)
```

Python : 3.13.13 - Windows

PySpark : 4.2.0

```
In [2]: # Si nécessaire (environ 1 minute à l'échelle 0.1) :
# !python3 generate_data.py --scale 0.1 --outdir ./data

import os
attendus = ["customers.csv", "products.csv", "orders.csv",
            "order_items.csv", "payments.json", "events.json"]
manquants = [f for f in attendus if not os.path.exists(os.path.join("data", f))]
print("Fichiers manquants :", manquants if manquants else "aucun - OK")
```

Fichiers manquants : ['customers.csv', 'products.csv', 'orders.csv', 'order_items.csv', 'payments.json', 'events.json']

```
In [7]: from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.driver.bindAddress", "127.0.0.1"))
```

```

    .appName("TP5_jointures")
    .master("local[*]")
    .getOrCreate()
spark.sparkContext.setLogLevel("WARN")
print("Spark", spark.version, "- session prete")

```

Spark 4.2.0 - session prete

Tableau de relevés

Remplissez ce dictionnaire **au fil du TP** (ré-exécutez la cellule après chaque mise à jour).
Graine 42 oblige : vos valeurs doivent être identiques à celles de vos voisins —
comparez-les, c'est un contrôle gratuit.

```

In [8]: releves = {
    "R1_count_commandes_clean":      None, # partie A
    "R2_count_apres_jointure":      None, # partie B1
    "R3_ca_sentinelle_avant_apres": None, # partie B3 (tuple)
    "R4_orphelines_count_et_part":  None, # partie C1 (tuple)
    "R5_ca_orphelines":             None, # partie C2
    "R6_entonnoir_sessions":        None, # partie D1 (tuple de 4)
    "R7_taux_etape_a_etape":        None, # partie D1 (tuple de 3, en %)
    "R8_purchase_orphelins":        None, # partie D3
  }
releves

```

```

Out[8]: {'R1_count_commandes_clean': None,
'R2_count_apres_jointure': None,
'R3_ca_sentinelle_avant_apres': None,
'R4_orphelines_count_et_part': None,
'R5_ca_orphelines': None,
'R6_entonnoir_sessions': None,
'R7_taux_etape_a_etape': None,
'R8_purchase_orphelins': None}

```

Partie A — Mise en place et reconstruction de commandes_clean (20 min)

On recharge les quatre tables « transactionnelles » et on reconstruit la vue nettoyée du TP 4. Rappel du piège : ~1 % des montants de `orders.csv` portent un suffixe « FCFA », ce qui force **toute la colonne** en `string`.

```

In [10]: orders = spark.read.option("header", True).csv("../data/orders.csv")
customers = spark.read.option("header", True).csv("../data/customers.csv")
products = spark.read.option("header", True).csv("../data/products.csv")
items = spark.read.option("header", True).csv("../data/order_items.csv")

orders.createOrReplaceTempView("commandes_brutes")
customers.createOrReplaceTempView("clients")
products.createOrReplaceTempView("produits")
items.createOrReplaceTempView("lignes_commande")

print("orders      :", orders.count())
print("customers   :", customers.count())

```

```
print("products      :", products.count())
print("order_items   :", items.count())
```

```
orders      : 50000
customers   : 5025
products    : 632
order_items : 112750
```

A1 — La vue commandes_clean

Recréez la vue du TP 4 : montant nettoyé (suppression de tout caractère non numérique) puis casté en `BIGINT`.

```
In [13]: # === À COMPLÉTER ===
spark.sql("""
CREATE OR REPLACE TEMP VIEW commandes_clean AS
SELECT order_id, customer_id, date_commande, statut, canal,
       CAST(regexp_replace(trim(montant_total_fcfa), '[^0-9]', '') AS BIGINT) AS
       CAST(regexp_replace(trim(frais_livraison_fcfa), '[^0-9]', '') AS BIGINT) AS
FROM commandes_brutes
""")

total_commandes = spark.table("commandes_clean").count()
relevés["R1_count_commandes_clean"] = total_commandes
print("Releve R1 :", total_commandes)
```

Releve R1 : 50000

Partie B — Jointures multi-tables contrôlées (35 min)

B1 — Le CA par région, avec la discipline du cours

Comptage avant, jointure, comptage après — puis une phrase d'explication.

```
In [15]: # === À COMPLÉTER ===
spark.sql("""
CREATE OR REPLACE TEMP VIEW ventes_regions AS
SELECT o.*, c.ville, c.region
FROM   commandes_clean o
JOIN   clients c ON o.customer_id = c.customer_id
""")

apres = spark.table("ventes_regions").count()
relevés["R2_count_apres_jointure"] = apres
print("Avant :", total_commandes, "| Apres :", apres)
```

Avant : 50000 | Apres : 50123

Question B1 : le comptage après jointure est-il égal au relevé R1 ? Expliquez la différence en une ou deux phrases (vous vérifierez votre hypothèse en partie C).

Votre réponse : Le comptage après jointure (50123) est supérieur au comptage avant jointure (50000). On peut dire sans risque de nous tromper que il y'a des commandes avec un customer_id qui n'est pas dans le table.

B1 (suite) — Le CA par région

Sur les commandes **livrées** uniquement, trié décroissant.

```
In [18]: # === À COMPLÉTER ===
spark.sql("""
SELECT region,
       SUM(montant_total_fcfa) AS ca_fcfa,
       COUNT(*) AS nb_commandes
FROM   ventes_regions
WHERE  statut = 'livrée'
GROUP BY region
ORDER BY ca_fcfa DESC
""").show(20, truncate=False)
```

region	ca_fcfa	nb_commandes
Dakar	4460736600	19183
Thiès	1226866600	5183
Diourbel	1098868300	4631
Saint-Louis	529929300	2252
Kaolack	370844300	1580
Ziguinchor	358834800	1446
Louga	237832200	1086
Kolda	196939800	752
Tambacounda	142948100	632
Matam	140844200	641
Fatick	125001200	592
Kédougou	84213500	411
Sédhiou	70421000	311
Kaffrine	65828700	280

B2 — Le top produits : quatre tables

Chaîne complète `lignes_commande` → `commandes_clean` → `clients` → `produits`.

```
In [23]: # === À COMPLÉTER ===
spark.sql("""
SELECT p.categorie, p.nom_produit,
       SUM(CAST(i.quantite AS INT) * CAST(i.prix_unitaire_fcfa AS BIGINT)) AS ca
FROM   lignes_commande i
JOIN   commandes_clean o ON i.order_id    = o.order_id
JOIN   clients c          ON o.customer_id = c.customer_id
JOIN   produits p          ON i.product_id = p.product_id
WHERE  o.statut = 'livrée'
GROUP BY p.categorie, p.nom_produit
ORDER BY ca DESC
LIMIT 10
""").show(truncate=False)
```

categorie	nom_produit	ca
Informatique	Teranga Home Informatique 0005	1272528500
Informatique	Hisense Informatique 0001	916772500
Téléphonie	Lenovo Téléphonie 0006	549314800
Électroménager	Itel Électroménager 0016	459170400
Maison & Cuisine	Nivea Maison & Cuisine 0002	343124100
Électroménager	Lenovo Électroménager 0031	278259400
Électroménager	Samsung Électroménager 0063	188863000
Informatique	HP Informatique 0042	180194300
Téléphonie	Samsung Téléphonie 0052	149497500
Informatique	Dakar Style Informatique 0035	145154200

B3 — L'agrégat sentinelle

Le CA total (livrées) **avant** et **après** la jointure d'enrichissement. S'ils diffèrent, une jointure a perdu ou dupliqué des lignes.

```
In [25]: # === À COMPLÉTER ===
ca_avant = spark.sql(
    "SELECT SUM(montant_total_fcfa) FROM commandes_clean "
    "WHERE statut = 'livrée'").first()[0]
ca_apres = spark.sql(
    "SELECT SUM(montant_total_fcfa) FROM ventes_regions WHERE statut = 'livrée'")

releves["R3_ca_sentinelle_avant_apres"] = (ca_avant, ca_apres)
print("CA avant :", ca_avant)
print("CA apres :", ca_apres)
print("Ecart      :", ca_avant - ca_apres)
```

CA avant : 9090757800

CA apres : 9110108600

Ecart : -19350800

Question B3 : l'écart observé est-il une **perte** ou une **duplication** ? Quel indice vous permet de trancher sans même regarder la partie C ?

Votre réponse : L'écart est négatif ($ca_avant - ca_apres = -19\,350\,800$ FCFA), ce qui signifie que le CA après transformation est supérieur au CA avant. Or une perte de données ne peut que diminuer ou laisser inchangé le total — jamais l'augmenter. L'augmentation observée indique donc une duplication de lignes (probablement causée par une jointure qui multiplie certaines commandes, par exemple si une commande est associée à plusieurs régions). Cet indice de signe suffit à trancher sans consulter la partie C.

Partie C — L'enquête : commandes orphelines (30 min)

C1 — Compter et vérifier la partition

La question d'audit du cours : *toutes les commandes ont-elles un client connu ?*

```
In [26]: # === À COMPLÉTER ===
spark.sql("""
CREATE OR REPLACE TEMP VIEW commandes_orphelines AS
SELECT o.*
FROM   commandes_clean o
LEFT ANTI JOIN clients c ON o.customer_id = c.customer_id
""")

nb_orphelines = spark.table("commandes_orphelines").count()
part = 100.0 * nb_orphelines / total_commandes
revels["R4_orphelines_count_et_part"] = (nb_orphelines, round(part, 3))
print(f"Orphelines : {nb_orphelines} ({part:.3f} % du total)")
```

Orphelines : 100 (0.200 % du total)

Le **test de partition** du cours — obligatoire avant d'aller plus loin : `count(semi) + count(anti) = count(gauche)` .

```
In [27]: # === À COMPLÉTER ===
nb_valides = spark.sql("""
SELECT COUNT(*) FROM commandes_clean o
LEFT SEMI JOIN clients c ON o.customer_id = c.customer_id
""").first()[0]

assert nb_valides + nb_orphelines == total_commandes, "partition cassee !"
print("Partition exacte verifiee :",
      nb_valides, "+", nb_orphelines, "=", total_commandes)
```

Partition exacte verifiee : 49900 + 100 = 50000

C2 — Caractériser avant de décider

Échantillon, concentration temporelle, enjeu financier — la démarche d'enquête du cours.

```
In [29]: # === À COMPLÉTER ===
# 1. Echantillon : format des customer_id en cause ?
spark.table("commandes_orphelines").show(10, truncate=False)

# 2. Concentration temporelle ?
spark.sql("""
SELECT date_format(date_commande, 'yyyy-MM') AS mois, COUNT(*) AS n
FROM   commandes_orphelines
GROUP BY 1 ORDER BY 1
""").show(30)

# 3. Enjeu financier ?
ca_orph = spark.sql(
```

```
"SELECT SUM(montant_total_fcfa) FROM commandes_orphelines").first()[0]  
releves["R5_ca_orphelines"] = ca_orph  
print("CA porte par les orphelines :", ca_orph, "FCFA")
```

order_id	customer_id	date_commande	statut	canal	montant_total_fcfa
frais_livraison_fcfa					
0000772	C920100	2025-07-26 21:11:42	livrée	web	31000
1000					
00002246	C959990	2025-07-21 06:23:22	livrée	mobile_app	22500
1000					
00003494	C928943	2025-09-08 18:42:31	livrée	web	394300
2000					
00003620	C933335	2026-05-17 22:59:45	livrée	web	62500
1000					
00004146	C942074	2026-05-28 22:47:24	en_cours	web	3500
3000					
00004178	C976155	2026-01-20 18:10:24	livrée	mobile_app	107000
1500					
00005236	C943889	2025-07-23 22:09:56	livrée	mobile_app	160500
3000					
00005372	C981741	2026-01-10 14:22:18	retournée	mobile_app	555600
2000					
00005851	C993572	2024-09-29 12:07:20	en_cours	mobile_app	51000
3000					
00006485	C920105	2026-02-19 08:38:49	livrée	mobile_app	1139500
1000					

only showing top 10 rows

mois	n
2024-07	3
2024-08	5
2024-09	4
2024-10	2
2024-11	2
2024-12	6
2025-01	3
2025-02	1
2025-03	3
2025-04	5
2025-05	4
2025-06	3
2025-07	6
2025-08	3
2025-09	4
2025-10	6
2025-11	2
2025-12	6
2026-01	6
2026-02	8
2026-03	5
2026-04	4
2026-05	5
2026-06	4

CA porte par les orphelines : 25202900 FCFA

Question C2 — la décision. En vous appuyant sur vos relevés R4 et R5, rédigez en 3–4 lignes la décision que vous prenez pour la suite des analyses (quarantaine ? exclusion documentée ? conservation en « région inconnue » ?) et sa justification.

Votre réponse : Les commandes orphelines représentent un faible pourcentage du total et portent un enjeu financier limité. Je décide de les exclure des agrégats régionaux principaux tout en les documentant clairement. Je conserve en parallèle un segment « région inconnue » pour éviter d'éliminer complètement l'information de transaction, mais je traite ces données en quarantaine afin qu'elles ne biaisent pas l'analyse métier.

Question piège : réécrivez l'anti-jointure avec `NOT IN`. Obtenez-vous le même compte ? Dans quel cas ces deux écritures divergeraient-elles ?

Votre réponse : Oui, sur ces données le compte est identique (`NOT IN : 100`, `LEFT ANTI : 100`). En revanche, ces deux formules peuvent diverger si la sous-requête renvoie des `NULL` : `NOT IN` devient alors indéterminé et peut exclure des lignes supplémentaires, tandis que `LEFT ANTI JOIN` reste le bon choix pour les cas où `customer_id` est `NULL` dans l'un des jeux de données.

```
In [41]: # === À COMPLÉTER ===
nb_not_in = spark.sql("""
SELECT COUNT(*) FROM commandes_clean
WHERE customer_id NOT IN (SELECT customer_id FROM clients)
""").first()[0]
print("NOT IN :", nb_not_in, "| LEFT ANTI :", nb_orphelines)
```

NOT IN : 100 | LEFT ANTI : 100

Partie D — L'entonnoir de conversion (40 min)

D1 — Les comptages par étape

On charge `events.json` (~330 000 événements à l'échelle 0.1) et on compte des **sessions distinctes** par étape — jamais des événements bruts.

```
In [32]: events = spark.read.json("../data/events.json")
events.createOrReplaceTempView("events")
print("Evenements :", events.count())
events.printSchema()
```

Evenements : 329976

```
root
|-- device: string (nullable = true)
|-- event_id: string (nullable = true)
|-- event_time: string (nullable = true)
|-- event_type: string (nullable = true)
|-- order_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- session_id: string (nullable = true)
|-- user_id: string (nullable = true)
|-- ville: string (nullable = true)
```

```
In [33]: # === À COMPLÉTER ===
funnel = spark.sql("""
SELECT
    COUNT(DISTINCT session_id)                AS sessions,
    COUNT(DISTINCT CASE WHEN event_type = 'view_product'
                        THEN session_id END)    AS vues,
    COUNT(DISTINCT CASE WHEN event_type = 'add_to_cart'
                        THEN session_id END)    AS paniers,
    COUNT(DISTINCT CASE WHEN event_type = 'purchase'
                        THEN session_id END)    AS achats
FROM events
""").first()

s1, s2, s3, s4 = funnel
releves["R6_entonnoir_sessions"] = (s1, s2, s3, s4)
print("Sessions :", s1, "| Vues :", s2, "| Paniers :", s3, "| Achats :", s4)
```

Sessions : 80000 | Vues : 60281 | Paniers : 33779 | Achats : 50000

Les **taux** : étape-à-étape (où est la fuite ?) et global.

```
In [34]: # === À COMPLÉTER ===
t_vue    = round(100.0 * s2 / s1, 1)
t_panier = round(100.0 * s3 / s2, 1)
t_achat  = round(100.0 * s4 / s3, 1)
t_global = round(100.0 * s4 / s1, 1)

releves["R7_taux_etape_a_etape"] = (t_vue, t_panier, t_achat)
print(f"vue {t_vue} % -> panier {t_panier} % -> achat {t_achat} %")
print(f"taux global : {t_global} %")
```

vue 75.4 % -> panier 56.0 % -> achat 148.0 %
taux global : 62.5 %

D2 — Segmenter par device puis par ville

Même entonnoir, ventilé. Le mobile (~78 % du trafic) convertit-il mieux ?

```
In [35]: # === À COMPLÉTER ===
spark.sql("""
SELECT device,
    COUNT(DISTINCT session_id)                AS sessions,
    COUNT(DISTINCT CASE WHEN event_type = 'purchase'
                        THEN session_id END)    AS achats,
    ROUND(100.0 *
        COUNT(DISTINCT CASE WHEN event_type = 'purchase'
                        THEN session_id END)
        / COUNT(DISTINCT session_id), 2)      AS taux_global
FROM events
GROUP BY device ORDER BY sessions DESC
""").show()

# Meme requete par ville (limitez aux 10 premieres par sessions) :
...

```

```
+-----+-----+-----+-----+
| device|sessions|achats|taux_global|
+-----+-----+-----+-----+
| mobile|   79344| 39017|      49.17|
| desktop|  46976| 10007|      21.30|
| tablette|   6401|   976|      15.25|
+-----+-----+-----+-----+
```

Out[35]: Ellipsis

D3 — Le pivot vers les transactions

`order_id` n'est renseigné que sur les événements `purchase` : c'est le pont entre comportement et transactions. **Contrôle** : tout `purchase` pointe-t-il vers une commande connue ?

```
In [43]: # === À COMPLÉTER ===
purchase_orphelins = spark.sql("""
SELECT COUNT(*)
FROM (SELECT * FROM events WHERE event_type = 'purchase') e
LEFT JOIN commandes_clean o ON e.order_id = o.order_id
WHERE o.order_id IS NULL
""").first()[0]

releves["R8_purchase_orphelins"] = purchase_orphelins
print("Evenements purchase sans commande :", purchase_orphelins)
```

Evenements purchase sans commande : 0

Synthèse de l'entonnoir (à rédiger). Quatre paragraphes courts : **constat** (la marche la plus fuyante, chiffres à l'appui), **hypothèse** (pourquoi ?), **recommandation testable**, **limites** (pensez aux ~30 % de sessions anonymes et à la nature synthétique des données).

Votre réponse : La marche la plus fuyante de l'entonnoir est le passage de la vue produit au panier : un nombre significatif de sessions consultent des produits sans ajouter quoi que ce soit au panier, ce qui montre une fuite forte au niveau de l'engagement initial.

Mon hypothèse est que l'interface ou l'offre produit n'est pas suffisamment captivante, ou que la sélection de produits ne génère pas assez de confiance pour déclencher l'acte d'achat. Les utilisateurs semblent explorer plus qu'ils ne s'engagent.

La recommandation testable est de mesurer l'impact d'un message ou d'une promotion ciblée sur les pages produit, et de comparer le taux de passage au panier avant/après. Il faut surtout garder le suivi par appareil et par ville pour voir si le mobile ou certaines villes sont plus sensibles.

Limite : le dataset est synthétique et environ 30 % des sessions sont anonymes, ce qui réduit la fiabilité des conclusions sur le comportement utilisateur global. Les tendances observées restent utiles, mais elles doivent être validées sur un jeu de données réel avant décision métier.

Partie E — Bonus : fonctions fenêtres (15 min)

E1 — Top 3 produits par région (classement puis filtre)

```
In [39]: # === À COMPLÉTER ===
spark.sql("""
CREATE OR REPLACE TEMP VIEW ventes_par_region_produit AS
SELECT c.region, p.nom_produit,
       SUM(CAST(i.quantite AS INT) * CAST(i.prix_unitaire_fcfa AS BIGINT)) AS ca
FROM   lignes_commande i
JOIN   commandes_clean o ON i.order_id    = o.order_id
JOIN   clients c         ON o.customer_id = c.customer_id
JOIN   produits p        ON i.product_id  = p.product_id
WHERE  o.statut = 'livrée'
GROUP BY c.region, p.nom_produit
""")

spark.sql("""
SELECT * FROM (
  SELECT region, nom_produit, ca,
         ROW_NUMBER() OVER (PARTITION BY region
                             ORDER BY ca DESC) AS rang
  FROM   ventes_par_region_produit
) WHERE rang <= 3
ORDER BY region, rang
""").show(30, truncate=False)
```

region	nom_produit	ca	rang
Dakar	Teranga Home Informatique 0005	632075700	1
Dakar	Hisense Informatique 0001	450315200	2
Dakar	Lenovo Téléphonie 0006	264197400	3
Diourbel	Teranga Home Informatique 0005	148520200	1
Diourbel	Hisense Informatique 0001	113036200	2
Diourbel	Lenovo Téléphonie 0006	66451100	3
Fatick	Teranga Home Informatique 0005	16928000	1
Fatick	Hisense Informatique 0001	14359400	2
Fatick	Samsung Électroménager 0063	6113900	3
Kaffrine	Teranga Home Informatique 0005	9724400	1
Kaffrine	Hisense Informatique 0001	7314000	2
Kaffrine	Lenovo Téléphonie 0006	5718000	3
Kaolack	Teranga Home Informatique 0005	46996900	1
Kaolack	Hisense Informatique 0001	35994400	2
Kaolack	Lenovo Téléphonie 0006	23238900	3
Kolda	Teranga Home Informatique 0005	26028200	1
Kolda	Hisense Informatique 0001	21917400	2
Kolda	Lenovo Téléphonie 0006	13324600	3
Kédougou	Teranga Home Informatique 0005	10051000	1
Kédougou	Hisense Informatique 0001	9006900	2
Kédougou	Lenovo Téléphonie 0006	7304800	3
Louga	Teranga Home Informatique 0005	31473500	1
Louga	Hisense Informatique 0001	23770400	2
Louga	Itel Électroménager 0016	14129000	3
Matam	Teranga Home Informatique 0005	17293800	1
Matam	Hisense Informatique 0001	14824400	2
Matam	Itel Électroménager 0016	8041000	3
Saint-Louis	Teranga Home Informatique 0005	59685200	1
Saint-Louis	Hisense Informatique 0001	51680600	2
Saint-Louis	Lenovo Téléphonie 0006	32543100	3

only showing top 30 rows

E2 — Évolution mensuelle du CA (LAG)

Le pic de décembre du fil rouge doit apparaître : c'est votre contrôle de vraisemblance.

```
In [45]: # === À COMPLÉTER ===
spark.sql("""
CREATE OR REPLACE TEMP VIEW ca_mensuel AS
SELECT date_format(date_commande, 'yyyy-MM') AS mois,
       SUM(montant_total_fcfa)                AS ca
FROM   commandes_clean
WHERE  statut = 'livrée'
GROUP BY 1
""")

spark.sql("""
SELECT mois, ca,
       ca - LAG(ca, 1) OVER (ORDER BY mois) AS delta
FROM   ca_mensuel ORDER BY mois
""").show(30)
```

mois	ca	delta
2024-07	293487800	NULL
2024-08	288891300	-4596500
2024-09	305448500	16557200
2024-10	321980200	16531700
2024-11	306182200	-15798000
2024-12	490298700	184116500
2025-01	327247800	-163050900
2025-02	288641000	-38606800
2025-03	318877600	30236600
2025-04	332621300	13743700
2025-05	367180800	34559500
2025-06	353753900	-13426900
2025-07	356315600	2561700
2025-08	396396800	40081200
2025-09	367504700	-28892100
2025-10	406067000	38562300
2025-11	366408000	-39659000
2025-12	634215100	267807100
2026-01	415160800	-219054300
2026-02	381986400	-33174400
2026-03	430291900	48305500
2026-04	433716900	3425000
2026-05	474679700	40962800
2026-06	433403800	-41275900

Relevés finaux et livrable

Ré-affichez le tableau complet : **aucune valeur ne doit rester à None** (R7 exclu si vous n'avez pas atteint le bonus — indiquez-le alors en commentaire).

```
In [46]: for cle, valeur in releves.items():
          print(f"{cle:35s} : {valeur}")

restants = [k for k, v in releves.items() if v is None]
print("\nRelevés manquants :", restantes if restantes else "aucun - OK")
```

```
R1_count_commandes_clean      : 50000
R2_count_apres_jointure       : 50123
R3_ca_sentinelle_avant_apres  : (9090757800, 9110108600)
R4_orphelines_count_et_part   : (100, 0.2)
R5_ca_orphelines              : 25202900
R6_entonnoir_sessions        : (80000, 60281, 33779, 50000)
R7_taux_etape_a_etape        : (75.4, 56.0, 148.0)
R8_purchase_orphelins        : 0
```

Relevés manquants : aucun - OK

Pont vers le livrable

Depuis la racine de votre dépôt :

```
git add notebooks/TP5_jointures.ipynb
git commit -m "TP5 : jointures, audit orphelines, entonnoir"
git push
```

Vérifiez sur github.com que le notebook s'affiche **avec ses sorties**.

Avant la séance 6 : repérez 2–3 jeux de données publics réels (data.gouv.sn, ANSD, Kaggle, data.humdata.org...) candidats pour votre projet individuel — le **jalón 0** (fiche de cadrage) sera lancé en séance 6. Lecture : Reis & Housley, chap. 6 (*Storage*).